

RESUMEN PROGRAMACIÓN TEMA 4

(Programación Orientada a Objetos, Excepciones y Arrays)

PROGRAMACIÓN ORIENTADA A OBJETOS

- **Clase:** abstracción de un concepto que posee datos y funcionalidad.
- **Objeto:** Instancia de una clase

Una clase es un *molde* con el cual se generan los objetos: define sus propiedades y su comportamiento.

Componentes de una clase

- **Atributos:** son las propiedades de los objetos de la clase.
- **Constructores:** procedimientos que se ejecutan en el momento de crear un objeto clase. Tienen el mismo nombre de la clase y pueden existir varios.
- **Métodos:** procedimiento u operaciones que comparten los objetos de clase.

Características de la P.O.O.

Abstracción: Un objeto es capaz de desempeñar una función independientemente del contexto en que éste es utilizado. Puede ser *Funcional* o *De Datos*.

Encapsulación: Se relaciona con la ocultación de la información, definiendo que partes de un objeto son visibles (public) y que partes son ocultas (private).

Herencia: Permite definir a partir de una clase, otras clases relacionadas, esto supone Especialización y Generalización. Su ventaja es que no es necesario repetir código ya que hereda de ella.

Las clases derivadas (hijas o subclases) heredan de la clase padre (o superclase).

Polimorfismo: Permite que objetos de un mismo super(tipo) puedan realizar una operación distinta.

Declaración de una clase

Son palabras reservadas que se anteponen a la declaración de la clase e indican la visibilidad de la clase.

Los **atributos** indican desde qué parte del código se puede acceder a la variable:

- public: método o atributo es siempre visible y accesible para todas las clases.
- private: Sólo visibles desde dentro de la propia clase.
- protected: Sólo visibles por la propia clase y las que heredan de ella.
- [nada]: la clase será accesible sólo desde clases que están en el mismo paquete (friendly).

Los **métodos** son bloques de código que realizan una tarea específica, estos pertenecen a una clase y podrá acceder a los atributos de la misma (salvo que sea estático). Los métodos tienen:

- Una lista de parámetros de entrada al método.
- Un valor devuelto.
- Puede tener también variables locales al método.

Los métodos pueden no tener ningún parámetro y/o no devolver ningún valor, y pueden ser llamados desde cualquier punto de la aplicación.

Constructores

Los constructores poseen unas características especiales:

- Se tienen que llamar igual que la clase.
- No tienen valor de retorno.
- Pueden existir varios, que se distinguirán por los parámetros que aceptan (sobrecarga).
- Debe existir al menos un constructor.
- Si no se codifica ningún constructor, se crea uno por defecto sin argumentos.
- Si la clase tiene algún constructor, el constructor por defecto no se crea.

Objetos

Un objeto es una instancia de una clase. Para poder utilizar un objeto hay que:

- Declarar una referencia a la clase.
- Crear un objeto mediante el operador new, invocando al constructor adecuado.

Para invocar un método de un objeto:

```
nombreObjeto.nombreMetodo(...)
```

Java proporciona una referencia al objeto con el que se está trabajando, esta referencia se denomina **This** que es el objeto que está ejecutando el método.

Métodos de clase

Los **sets** y **gets** son la forma de acceder a los atributos de una clase. Generalmente se usan con los atributos privados, ya que a los públicos se puede acceder directamente sin tener que acudir a ellos. Los métodos "**getters**" y "**setters**" proporcionan acceso a los atributos privados ya que no serían accesibles desde otras clases, además los set controlarán que no se den a los atributos valores no válidos.

El método **toString** es un método que tienen todos los objetos Java. Devuelve un String con la información de los atributos del objeto.

En Java, si se comparan dos objetos con el operador `==` se están comparando las referencias, no el contenido del objeto. Por defecto cuando creamos una nueva clase, ya tiene un método `equals`, pero compara, las referencias.

Elementos estáticos

Las variables static son compartidas por todos los objetos de la clase, el acceso a estas se puede realizar desde la clase o desde los objetos.

Para llamar a un método estático:

- Si se invoca desde la clase en la que se encuentra definido, basta con escribir su nombre.
- Si se le invoca desde una clase distinta, debe anteponer a su nombre, el de la clase en la que se encuentra seguido de un punto(.).

Restricciones métodos estáticos:

- Solo pueden acceder a variables y métodos estáticos de la misma clase.
- No pueden acceder a variables y/o métodos no estáticos directamente, tiene que crear primero un objeto.
- Los métodos de instancia sí pueden acceder a variables y métodos estáticos.
- Un constructor no puede ser estático.
- No puede acceder a `this`.

Una variable **final** no puede cambiar su valor a lo largo de la ejecución del programa, puede ser considerada una constante.

Deben ser inicializadas en su declaración.

Por convenio el identificador de una variable **final** se escribe en mayúsculas.

Es lógico definir las constantes como static, de forma que sean compartidas por todos los métodos y objetos que se creen. Ejemplo:

```
final static double PI = 3.141592;
```

Convenciones utilizadas en Java

- Los nombres de las clases empiezan por mayúscula.
- Los nombres de las variables, métodos y objetos comienzan por minúscula.
- Si contienen varias palabras se usa CamelCase.
- Los nombres de las constantes son en mayúscula.
- El código se tabula para una mejor compresión.

EXCEPCIONES

Una excepción es un evento que ocurre durante la ejecución de un programa, interrumpiendo el flujo normal de las instrucciones. Suelen estar asociadas a errores en **tiempo de ejecución**.

Ventajas de usar excepciones:

- Separar el código general del proceso de errores.
- Propagar errores hasta encontrar el manejador adecuado.
- Agrupar y diferenciar tipos de errores.
- Mejor legibilidad de código.

Excepciones en Java

Cuando se produce un error en tiempo de ejecución, se crea un objeto (de tipo Exception) con información relativa al error:

- Tipo de excepción.
- Estado del programa.

Una vez creada la excepción pueden ocurrir dos cosas:

- El programa se detiene y muestra el error (comportamiento no deseado).
- Existe código preparado para gestionar el error (comportamiento deseado).

El código destinado a detectar y tratar excepciones se denomina **manejador (handler)**.

Propagación de excepciones

Cuando se produce una excepción, puede ser tratada en el mismo método que la genera o propagarse hacia otros métodos “superiores” hasta llegar al main.

Manejo de excepciones

Cuando se produce una excepción se puede manejar de dos formas:

- Tratarla directamente.
- Lanzarla al método inmediatamente “superior” (**solo si no es el main**).

Si un método lanza una excepción, hay que indicarlo en su definición mediante la palabra reservada **throws** seguida del tipo de excepción.

Las excepciones se capturan en bloques:

- **try { ... }** incluye las instrucciones que puede generar la excepción.
- **catch (TipoExcepcion nombreVariable) { ... }** contiene las instrucciones que se ejecutarán en caso de generarse la excepción.
- **finally { ... }** instrucciones que se ejecutarán independientemente de que se genere o no la excepción.

La clase Exception

Todas las excepciones en Java heredan de la clase Exception.

Comparten el método printStackTrace que muestra información de la pila de llamadas que provocó la excepción.

Las excepciones del tipo RuntimeException no estamos obligados a capturarlas y tratarlas, el resto de excepciones **SI**.

Podemos crear clases excepciones que hereden de Exception y lanzarlas manualmente con **throw**.

ARRAYS

Los arrays surgen de la necesidad de agrupar datos de un programa en una estructura de datos. Un **array** (vector) es una colección ordenada de elementos del mismo tipo, donde cada elemento está asociado a un índice o posición que ocupa (espacios en la memoria). Se determina por:

- El tipo de sus elementos.
- El número de elementos.

Para utilizar un array o vector hay que realizar tres operaciones:



Declaración

Se pueden declarar de dos formas diferentes: `tipo[] nombre;` o `tipo nombre[]`. Donde tipo indica si es int, double, String, un objeto y nombre es un identificador que nombra al array.

Creación

Se indica el número de elementos del vector para que se pueda reservar la cantidad de memoria necesaria para contener todos sus elementos.

Inicialización

Si no se especifica ningún valor, los elementos de un vector se inicializan automáticamente a unos valores predeterminados (variables numéricas a 0, objetos a null y booleans a false).

Acceso y recorrido de Arrays

Para acceder al valor de un elemento vector, se utiliza el nombre del vector, seguido de un subíndice entre corchetes.

Si se intenta acceder a un elemento del vector, con un subíndice fuera de rango, Java lanzará una excepción del tipo: `ArrayIndexOutOfBoundsException`.

Para asegurarnos de no exceder el final de un vector usamos el atributo `length` para verificar su longitud.

```
//devuelve el nº de elementos del vector
int numeroDeElementos = nombreVector.length;
```

Siempre que se necesite realizar un tratamiento sobre todos los elementos de un vector realizaremos un recorrido del vector mediante un bucle for:

```
for (int i = 0; i < nombreVector.length; i++) {
    // Realizar el tratamiento que se desee con
    // el elemento vector[i]
}
```

Para simplificar esta codificación podemos usar la estructura **for each**:

```
for (String nombre : vectorNombres) {
    System.out.print(nombre);
}
```

Desde un for each no se pueden modificar los valores de un array.

Arrays como parámetros

Un vector puede pasarse como parámetro a un método. El paso siempre es por **referencia**, es decir, cualquier cambio que se haga en un método sobre los elementos de un vector se conservarán al abandonar dicho método.

- Al pasar un vector como parámetro solo se debe indicar el nombre del vector.
- En el método si se indica que es un vector con los corchetes `[]`.

Ordenación de elementos

Arrays.sort ordena por defecto de forma ascendente.

Para ordenar un array de objetos la clase tiene que implementar la interfaz **Comparable**, por lo que debe tener el método **compareTo**.

Otros métodos de arrays

equals devuelve true si los dos arrays comparados son el mismo, es decir, compara las referencias.

clone devuelve un array nuevo con los mismos datos que el array del que se clona.